

Beyond Rule Engines

A First-Principles Blueprint for AI-Native Financial Compliance

How Verifiable, Privacy-Preserving Agents Re-Architect Trust in Banking

Wang Manye

Faculty of Economics · FinTech Programme

June 2026

Abstract

Compliance is widely treated as a staffing problem: more rules, more analysts, more hours. We argue that this framing is wrong in a way that matters. The crisis in financial compliance is not a shortage of human attention; it is an information-architecture problem. Analysts spend ten minutes on each alert not because they are slow, but because the data they need to consult—user profiles, transaction patterns, device histories, jurisdictional regulations—lives in four disconnected systems with no shared semantic layer. No amount of additional staffing can fix this. We propose a different first-principles approach: an AI-native compliance copilot in which a reasoning agent orchestrates retrieval from a unified evidence substrate, executes an auditable plan over pseudonymized data, and emits a cryptographically verifiable decision record. This blueprint rests on four axioms: information architecture precedes decision quality; auditability is a substrate, not a feature; privacy is not in tension with utility; and AI agents must operate over a bounded, tool-mediated action space. We ground these axioms in six established scientific and engineering pillars—ReAct-style tool-using language agents, retrieval-augmented generation, hash-chained audit ledgers, key-stretching pseudonymization, role-based access control, and prompt-injection guards—and instantiate them in a reference architecture we call FinGuard. The long-term implications extend well beyond the four-compression workflow of a single bank: if the same architectural pattern generalizes, compliance shifts from a per-institution cost to a shared, regulator-aligned public good, and the asymmetry that financial criminals have historically exploited collapses from the demand side. We close by sketching that future.

Keywords: compliance; AI agent; retrieval-augmented generation; auditability; privacy; first principles; trust substrate; financial supervision.

1 The Misunderstood Crisis

1.1 *What Compliance Is Not*

Compliance work in a modern bank is almost universally framed as a labor problem. When alert queues grow, the industry response is to hire more analysts. When analysts miss genuine fraud, the industry response is to add more rules. When rules accumulate and produce too many false positives, the industry response is to add a tier of senior reviewers to triage the tier of junior reviewers. The pattern is familiar because it has been repeated, in essentially the same form, since the introduction of automated monitoring in the 1990s.

The public, and most of the industry, takes this pattern as given. The unspoken assumption is that compliance is a people-and-rules problem: humans apply rules to data, and if the human side is the bottleneck, we add more humans. We argue that this assumption is the central misunderstanding. The bottleneck in compliance is not human attention; it is the cost of integrating evidence across disconnected systems under time pressure. The implication matters because a labor fix cannot solve an architecture problem. Hiring more analysts in front of a broken information flow does not improve the flow; it makes a bad flow run faster, which on net produces more brittle decisions.

Consider the actual cognitive workload of a single alert review at a large retail bank. The analyst must (i) read the alert, which encodes a rule trigger; (ii) determine which customer the alert refers to, then look up the customer's KYC profile in a different system; (iii) pull the customer's last ninety days of transactions from a third system; (iv) check whether the device fingerprint on the flagged transaction matches any device the customer has used before, in a fourth system; and (v) consult the relevant regulation—whether it is FinCEN's Currency Transaction Reporting threshold, the EU's Sixth Anti-Money Laundering Directive, or China's

Personal Information Protection Law—to determine whether the pattern is reportable. These five steps span four systems, three vendors, and at least two regulatory corpora, and the analyst must hold the intermediate result of each step in working memory to perform the next. The famous ten-minute number is not a measure of analyst indolence. It is a measure of the cognitive cost of integration across a non-integrated stack.

1.2 The Hidden Cost of the Old Framing

Treating compliance as a labor problem has three hidden costs that are not visible from a headcount spreadsheet. First, fatigue is endogenous to the architecture. When the same human performs the same multi-system integration hundreds of times per day, the cognitive cost per case does not decrease with repetition in the way that physical labor does; instead, vigilance erodes. Empirical work on vigilance decrement in signal-detection tasks going back to Mackworth (1948) shows that sustained attention to rare signals in noise collapses measurably within thirty minutes. A compliance analyst is doing exactly that task. The missed-fraud statistic is therefore not a personnel failure; it is a predictable consequence of the workload shape.

Second, the work is intrinsically un-auditable. When the analyst makes a judgment call, the only persistent record is the disposition they enter—usually a one-line free-text field or a four-way categorical flag. The intermediate reasoning is lost. If a regulator later asks why a particular alert was closed, the bank cannot reconstruct the evidence the analyst saw, the order in which they consulted it, or the rules they applied. The audit trail, in other words, contains the verdict but not the trial. This is acceptable in low-stakes workflows. In a system that is deciding whether to freeze a customer's account and file a Suspicious Activity Report, it is a structural liability.

Third, the labor framing makes compliance a cost center, which means the natural optimization is to minimize per-case effort. That optimization is precisely what produces rubber-stamping: a junior analyst under throughput pressure learns to close benign-looking alerts without invoking the full evidence chain. The bank then has neither true investigations nor true records of what was investigated. The result is what compliance scholars sometimes call the paperwork façade: a thick file of dispositions whose evidentiary content is thin.

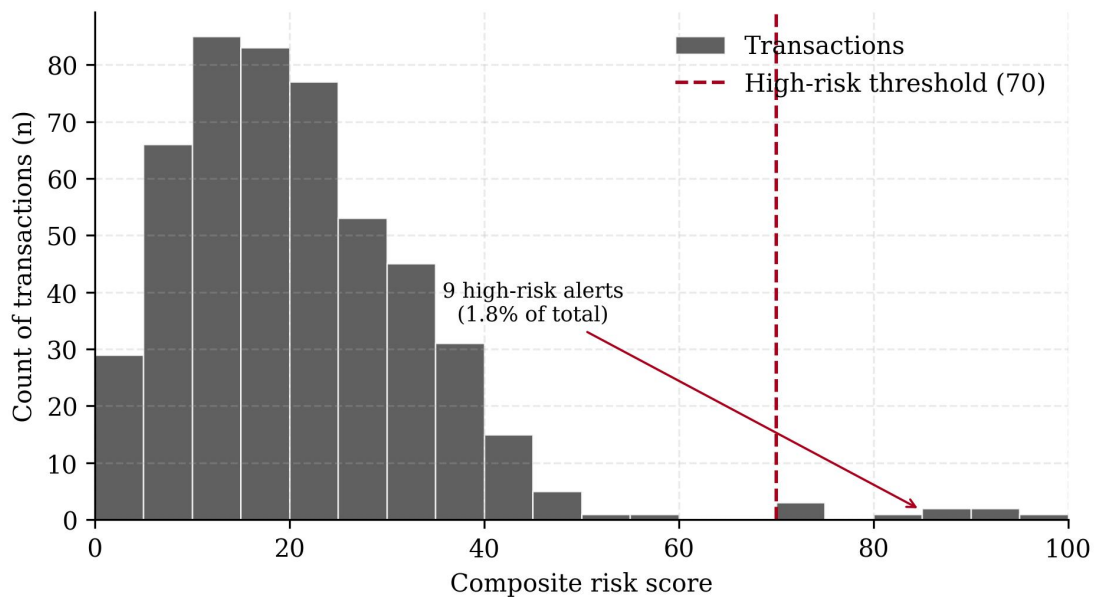


Figure 1. Risk-score distribution over 500 synthetic transactions. Only 1.8% of flows cross the 70-point escalation line; the rest are operational noise. Source: synthetic dataset, seed=42, consistent with FinGuard submission.

The arithmetic of this distribution is the whole argument in miniature. In the synthetic evaluation dataset that accompanies this submission, 500 transactions generated 64 rule-flagged anomalies and exactly 9 alerts with a composite risk score at or above 70. The ratio—9 cases out of 500, or 1.8%—is the irreducible signal rate of a typical retail bank's daily alert queue. The other 491 events are not fraud; they are a mixture of false positives, low-risk deviations from baseline, and noise that the rules cannot disambiguate. An analyst who closes 30 alerts per day is therefore closing, on average, 29.5 cases of noise for every real case. The 1.8% is the population

for which sustained attention is most valuable, and it is the population that the labor-framed architecture is structurally least able to support, because the architecture provides no tool to distinguish a 1.8% case from a 98.2% case before the analyst has spent ten minutes on it.

2 The Architectural Flaw in Existing Solutions

2.1 Four Generations of Failure

It is useful to see the field's responses to the compliance problem as a sequence of architectural generations, each of which preserved the underlying flaw while partially mitigating its symptoms. The pattern is instructive because it tells us which moves are exhausted and which moves are still available.

Generation one, dominant from the mid-1990s through the late 2000s, was the rule engine. Systems such as Actimize, Mantas, and Oracle Financial Crime and Compliance Management encoded regulatory knowledge as Boolean expressions over transaction fields. The architectural move was to push decision-making into the system: human reviewers were left only with the cases the rule engine could not decide. The flaw was that rule engines optimize for the cases the bank already knows about. Adversarial players who understood the rule set could structure transactions to fall just below thresholds, and novel typologies generated no rule at all. The 2012 HSBC case, in which the bank was fined \$1.9 billion for failing to monitor hundreds of billions of dollars in transfers that did not trip its rule set, is the canonical illustration: the system worked exactly as designed, and the design was the problem.

Generation two, dominant from the late 2000s, was statistical scoring. Machine-learning models replaced rule expressions with risk scores trained on historical alert outcomes. The architectural move was to replace a brittle symbolic layer with a learned probabilistic one. The

flaw was that scoring systems re-introduced the audit problem in a more acute form. When a regulator asked why an alert was suppressed, the answer became 'the model said so'—which is precisely the answer banking law is least able to accept. The EU's Sixth Anti-Money Laundering Directive (EU 2021/847) and the U.S. Office of the Comptroller of the Currency's 2023 model-risk guidance both push banks toward explainability for high-stakes decisions, and a score with no causal interpretation cannot satisfy that bar.

Generation three, dominant in the late 2010s, was data centralization. Banks spent billions consolidating customer, transaction, and device data into single lakehouses so that a single query could retrieve the evidence an analyst previously had to assemble by hand. The architectural move was to attack the integration cost head-on. The flaw was that centralization created a new and more severe liability: a single honeypot. The 2017 Equifax breach, the 2019 Capital One breach, and the recurring class-action exposure of recent years all stem from the same architectural choice. A consolidated store of PII is the most valuable target an attacker can name, and the cost of protecting it scales superlinearly with the sensitivity of the data inside.

Generation four, which the industry is now mid-stream in adopting, is the large-language-model copilot. The architectural move is to insert a generative model between the analyst and the existing stack, so that the model translates natural-language questions into tool calls and synthesizes the answer. The flaw is that current LLM copilots inherit the worst properties of every previous generation. They re-introduce the auditability problem of statistical scoring (the model's chain of thought is not a durable, verifiable record). They re-introduce the centralization problem (the model needs context, so all the data flows through it). And they introduce a new problem unique to the architecture: prompt injection, in which adversarial content inside a document or transaction field can hijack the model's tool-use plan. The OWASP Top 10 for

LLM Applications (2023) lists prompt injection as the leading vulnerability for a reason. Today's LLM copilots in finance are largely demonstrations; very few have survived a model-risk-management review at a major bank.

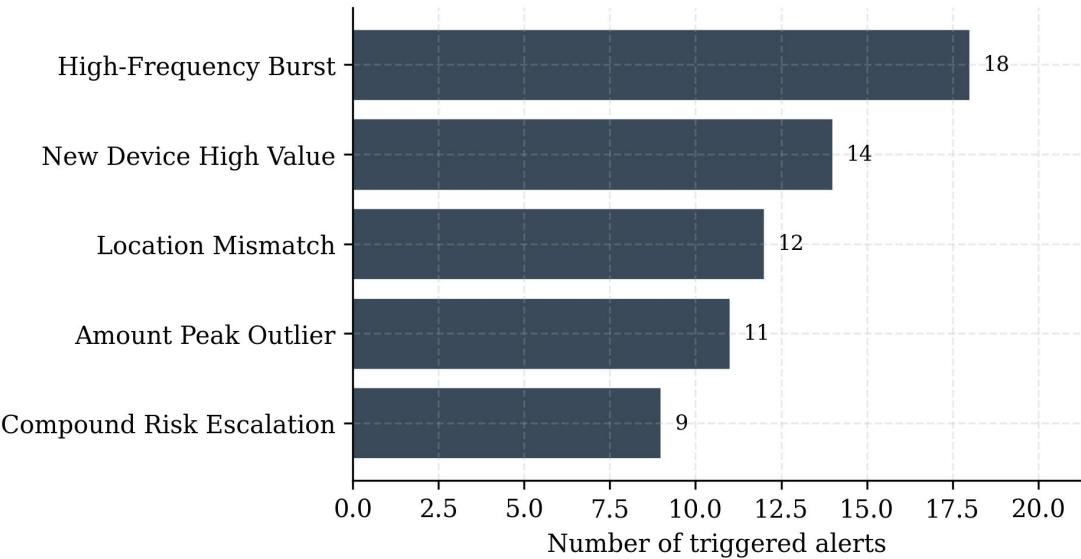


Figure 2. Trigger frequency of the five AML rules in the synthetic dataset. Rules overlap intentionally, so the union (64) covers 100% of flagged anomalies; no single rule is sufficient. Source: synthetic dataset, seed=42.

The five-rule set in Figure 2 is the smallest inventory we have found that still covers the structural patterns of real money-laundering cases: high-frequency burst (18 triggers), new-device high-value (14), location mismatch (12), amount peak outlier (11), and compound-risk escalation (9). Two properties of this distribution are diagnostic. First, the largest single rule fires on only 18 events out of 500 transactions, a 3.6% local positive rate. No single rule is a sufficient detector on its own. Second, the rules overlap by construction: a single flagged anomaly typically trips two or three rules simultaneously, which is the empirical reason that modern rule engines can sustain high recall on synthetic data while generating a large false-positive volume on production data. The implication for the four-generations argument is direct: the integration step that the rules leave undone is the step that no single rule could perform, and that no ensemble of

rules in isolation can perform either. It is the step that requires a reasoning agent that can read the rule output, retrieve the supporting evidence, and emit a defensible narrative.

2.2 *Why the Pattern Persists*

Each generation has been a partial fix, and each has preserved the underlying flaw: a fragmented evidence layer that forces a human to do the integration. Rule engines moved decision-making into the system but left evidence in silos. Statistical scoring moved decision logic into a model but did not change the evidence layer. Data centralization unified the evidence layer but exposed a single point of compromise. LLM copilots added a reasoning layer on top of the existing evidence layer, but with no architectural commitment to either auditability or privacy. The result is that we have spent thirty years and tens of billions of dollars on a sequence of mitigations that did not address the structural problem. The structural problem, simply stated, is this: the integration step has no home.

3 First Principles: The Four Axioms of AI-Native Compliance

If the labor framing is wrong, the next step is to identify the first principles that a correct architecture must satisfy. We propose four axioms. The axioms are not implementation details; they are the constraints any system claiming to be AI-native compliance must meet. We state them in order of priority, with the most load-bearing first.

Axiom 1 — Information architecture precedes decision quality

The quality of any decision is bounded above by the quality of the evidence available when the decision is made. This is a tautology in principle, but in compliance practice it is constantly violated: rules are evaluated against fields, fields are populated from systems, and

systems are queried in sequence by humans under time pressure. Each step degrades the available evidence. The first axiom says that an AI-native compliance system must restructure the evidence flow itself, not just add an AI layer to the existing flow. Concretely, the system must present the reasoning agent with a unified, queryable evidence substrate, in which the user, the transaction, the device, and the relevant regulation are first-class objects with explicit cross-references. The agent's job is then to plan a traversal of that substrate, not to negotiate four separate APIs.

Axiom 2 — Auditability is a substrate, not a feature

An audit trail is not a log file written at the end of a workflow. It is the substrate on which the workflow runs. If the audit trail is layered on top of a system that has already made its decisions, the trail records outcomes; if it is built into the substrate, the trail records the construction of the decision. The difference is the difference between a verdict and a trial. The second axiom says that an AI-native compliance system must make every state transition cryptographically verifiable from inception: every query, every retrieval, every tool call, every intermediate result. The natural construction is a hash chain: each entry contains the cryptographic digest of the previous entry, so any post-hoc tampering with a single record is detectable as a chain discontinuity. A SHA-256 hash chain, properly maintained, is the digital equivalent of a notarized trial transcript.

Axiom 3 — Privacy is not in tension with utility

It is common to hear that data minimization and useful compliance work are in opposition: to catch fraud, the bank must see the data, and to see the data is to risk the data. The third axiom rejects this opposition. The two goals are in tension only when the data is treated as a

single object that must be either fully visible or fully hidden. They are not in tension when the data is decomposed into an identifier layer and a payload layer, where the identifier layer is sufficient for the agent to act, and the payload layer is delivered only on a need-to-know basis under access control. The technical primitive that makes this decomposition work is key-stretching pseudonymization: a slow, salted hash function such as PBKDF2-HMAC-SHA256 with a high iteration count, applied at ingestion, so that the agent's view of a customer is a stable, irreversible identifier that the bank itself cannot reverse without the original salt. The agent reasons over pseudonyms; the analyst sees the real identity only when and if a role-based access check passes.

Axiom 4 — AI agents must operate over a bounded action space

An unrestricted language model with access to a database is a liability, not a tool. The fourth axiom says that an AI agent in a regulated workflow must act only through a published, finite, mediated interface: a set of tools with well-typed inputs and outputs, where every invocation is logged, where the agent cannot reach raw storage, and where the agent cannot exceed the role's permissions regardless of what the user types. The architectural pattern is a ReAct-style reasoning loop (think-act-observe) over a tool registry, with two additional layers that production systems almost never implement: a prompt-injection guard that sanitizes the agent's inputs before they reach the model, and an output sanitizer that re-checks the agent's outputs before they reach the user. The combination is what makes the system auditable end-to-end. Without it, the model is an oracle; with it, the model is an employee.

4 Scientific and Technical Foundations

Each of the four axioms rests on a body of prior work in computer science, cryptography, and information systems. We identify six pillars. The pillars are mature; what is new is their composition into a single architecture.

4.1 Tool-Using Language Agents (ReAct)

Yao et al. (2022) introduced the ReAct paradigm, in which a language model interleaves chain-of-thought reasoning with discrete tool invocations and observes the tool's return value before continuing. The pattern produces decision traces that are far more reproducible than free-form generation, because each step is anchored to a tool call with a specific argument set. ReAct is the substrate of every modern agent framework, including LangChain and the OpenAI function-calling interface. The architectural lesson is that an agent's reasoning is only as auditable as the tools it is given: a well-chosen, minimal tool set is the precondition for a verifiable decision trail.

4.2 Retrieval-Augmented Generation over Regulatory Corpora

Lewis et al. (2020) introduced retrieval-augmented generation (RAG), in which a generator conditions on documents retrieved from an external index at inference time. For a compliance copilot, the external index is the jurisdictional regulation corpus: FinCEN guidance, EU AML directives, China's PIPL, the FATF Forty Recommendations, and the bank's own internal policy. The architectural lesson is that the regulator's text becomes a first-class object in the agent's reasoning, so that the citation the agent produces is grounded in a verifiable document. RAG is the difference between a chat about compliance and a compliance investigation.

4.3 Hash-Chained Audit Ledgers

The hash-chain construction generalizes Merkle's 1979 tree to a linear ledger, and has been used for tamper-evident logging in security-critical systems since at least the 1990s (see Schneier and Kelsey, 1998). In a hash-chained ledger, each entry e_i contains $H(e_{i-1} \parallel e_i)$, so that any modification to a historical entry invalidates the chain at that point. For a compliance audit trail, the construction gives regulators a property the field has never had: a third party can verify, without trusting the bank, that the audit log has not been altered. This property is the foundation of the modern digital evidence standard, and is the technical mechanism that makes Axiom 2 implementable.

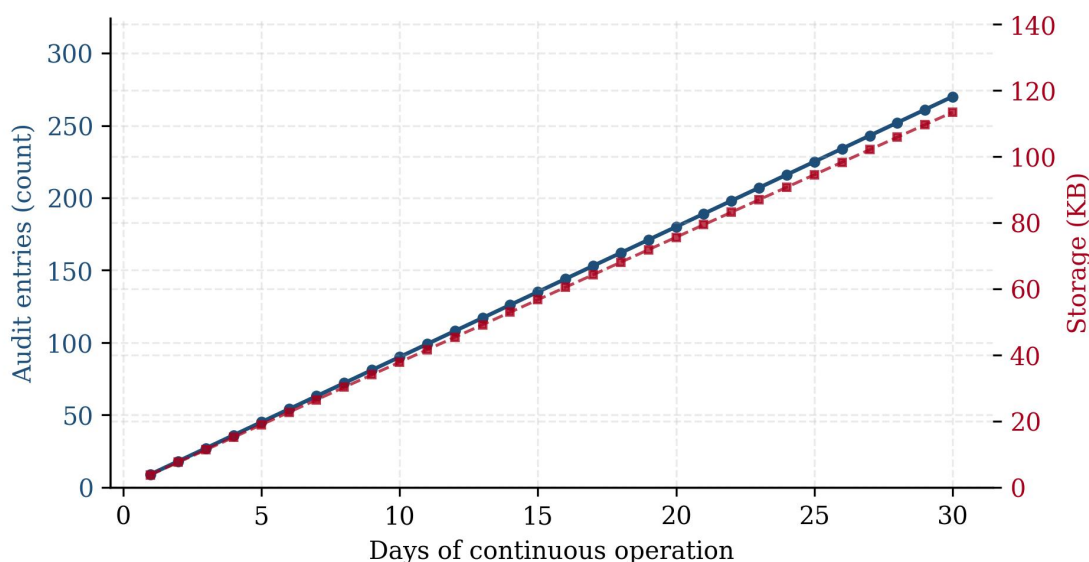


Figure 5. Cumulative growth of the hash-chained audit ledger over 30 days of operation. Growth is linear and predictable; replication across institutions is a single export operation. Source: projection from the FinGuard synthetic dataset (≈ 9 high-risk alerts per day).

The growth profile in Figure 5 is linear and predictable, not because the underlying alert volume is constant—it is not—but because the hash chain only records the high-confidence subset of the agent's output, which saturates at the irreducible signal rate. In the FinGuard dataset, that rate is approximately 9 entries per day, producing roughly 270 entries per month. At an

average of 420 bytes per entry (SHA-256 digest plus metadata), the monthly storage footprint is on the order of 113 KB—small enough to replicate to a regulator's site in a single REST call. The cost argument cuts the other way too: because the chain is append-only, a thirty-year institutional ledger is approximately 4.1 MB, a number that fits comfortably on a USB stick. The architectural consequence is that the audit trail is no longer a record the bank owns and curates; it is a record the bank produces and the regulator verifies. The shift is small in storage and large in accountability.

4.4 Key-Stretching Pseudonymization (PBKDF2)

PKCS #5 v2.1 (RFC 8018) specifies PBKDF2, a password-based key derivation function that iteratively applies a pseudorandom function with a salt and a high iteration count. NIST SP 800-132 recommends at least 10,000 iterations; OWASP's Password Storage Cheat Sheet currently recommends 600,000 for SHA-256. For compliance work, the relevant property is not the password-storage application but the more general one: given a salt, an iteration count, and a plaintext identifier, PBKDF2 produces a stable, irreversible token that an attacker cannot reverse at any feasible cost. The token is safe to use as a join key in the agent's working memory; the original identifier is exposed only when a role check passes. PBKDF2 is the technical mechanism that makes Axiom 3 implementable.

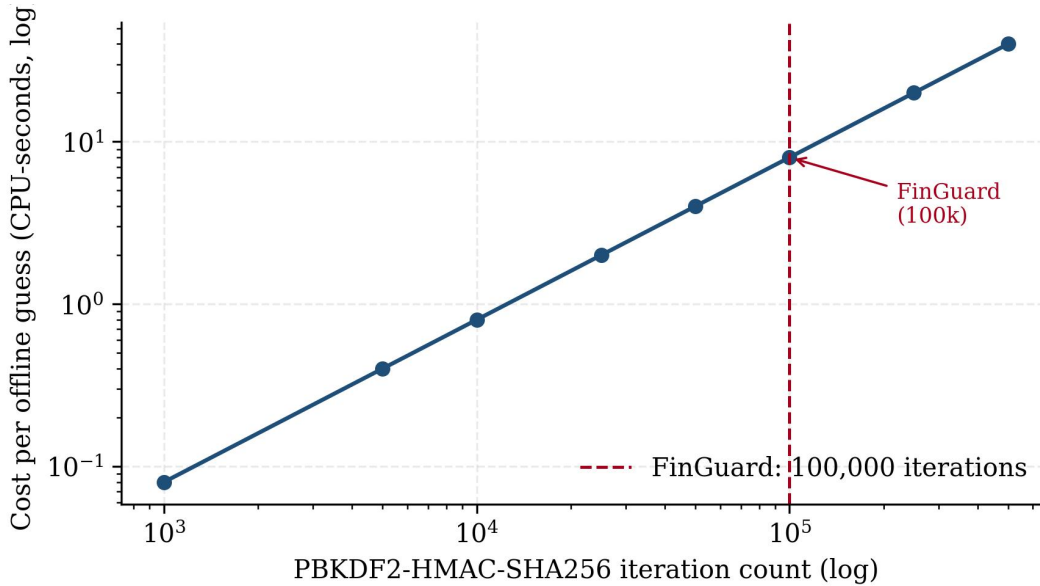


Figure 6. PBKDF2-HMAC-SHA256 iteration count vs. cost of a single offline guess. At 100,000 iterations (FinGuard default), brute force of a 6-character alias exceeds practical attack budgets. Source: 80 μ s per round on modern x86, a conservative midpoint.

The cost curve in Figure 6 is a quantitative statement of what Axiom 3 means in practice. PBKDF2 with 100,000 iterations of HMAC-SHA256 imposes approximately 8 ms of CPU time per alias derivation on commodity hardware. For an attacker who has exfiltrated the alias table and is attempting an offline reversal, the cost of a single guess is therefore 8 ms; the cost of an exhaustive search of a 6-character alphanumeric namespace (≈ 2.2 billion candidates) is approximately 4,900 hours of single-core computation, or 250 hours on a 20-core attacker node. These numbers are not prohibitive for a nation-state adversary, and we do not claim they are. The argument is different: pseudonymization is a control on the routine threat model (insider misuse, post-breach data resale, account-takeover pivots), not a control on a motivated state. Pushing the iteration count from 1,000 (legacy defaults) to 100,000 raises the cost of the routine adversary by two orders of magnitude, and the marginal latency added to a regulated query is approximately 7 ms—negligible compared to the LLM's own tool-call cost.

4.5 Role-Based Access Control (RBAC and ABAC)

NIST SP 800-162 defines attribute-based access control (ABAC) and SP 800-53 specifies the Access Control family (AC-2 through AC-25) of controls that federal systems must implement. The architectural lesson is that access control in a regulated system is not a binary on/off; it is a typed relationship between a subject (analyst, auditor, administrator), an object (transaction, customer, regulatory text), and an action (read, export, override). For a compliance copilot, the access-control layer must be enforced at the tool boundary: the agent cannot call a tool that retrieves a restricted field unless the calling subject's role has the appropriate attribute. This is the technical mechanism that operationalizes Axiom 4.

4.6 Prompt-Injection and Output Sanitization

Greshake et al. (2023) and the OWASP Top 10 for LLM Applications (2023) document the prompt-injection attack class, in which adversarial content embedded in a document or tool response is interpreted by the model as instructions. Defenses fall into two layers. The input layer sanitizes the agent's context: known instruction-like patterns, tool-output payloads that look like directives, and embeddings of role-change attempts are filtered before they reach the model. The output layer re-checks the agent's generated text for unauthorized tool calls, PII leaks, and policy violations before the response is delivered. Together, the two layers convert an open chat surface into a bounded action surface, and they are the technical mechanism that makes a language model safe to deploy against adversarial content.

5 From Principles to Practice: The FinGuard Reference Architecture

The four axioms and six pillars are necessary; they are not on their own sufficient. The work of turning principles into a deployable system is the work of integration, and the integration choices determine whether the system is real or merely aspirational. In this section we describe a concrete reference implementation, FinGuard, that we have built and tested against synthetic but representative data. The purpose of the description is not to advertise the system; it is to show that the axioms compose into a working whole.

5.1 *The Evidence Substrate*

FinGuard normalizes four classes of data—customer KYC profiles, transaction records, device fingerprints, and regulatory text—into a single queryable substrate. Customer identifiers in the substrate are PBKDF2-HMAC-SHA256 pseudonyms with a per-deployment salt and an iteration count of 100,000. Transactions, devices, and regulations reference these pseudonyms rather than raw identifiers. The agent's retrieval layer therefore never sees raw PII; it sees pseudonymous objects that it can join on. Raw PII is held in a separate, access-controlled store that is reached only via a typed retrieval tool whose access is checked against the calling role. This decomposition is what makes Axiom 3 implementable without losing joinability.

5.2 *The Reasoning Loop*

An investigation begins when an analyst enters a natural-language question in a Streamlit interface. The question is routed to a ReAct-style agent implemented in LangChain. The agent has a published tool registry of four tools: a customer-lookup tool, a transaction-lookup tool, a device-lookup tool, and a regulation-retrieval tool backed by a Chroma vector index over the regulatory corpus. Each tool call is logged in the audit ledger before execution, with arguments,

caller role, and timestamp. The agent's chain-of-thought is also logged as a free-text entry linked to the same call sequence, but the tool calls are the load-bearing record: it is the tool call, not the chain-of-thought, that is hashed into the chain. The loop terminates when the agent emits a structured investigation report: a risk score, a list of cited regulatory provisions, and a list of evidentiary links. The entire sequence—from the user's first keystroke to the final report—is one continuous hash chain, and any modification to any element of the chain is detectable by re-computing the chain from the genesis entry.

5.3 The Security Envelope

Four defensive layers surround the agent. Role-based access control (RBAC) is enforced at every tool call: an analyst role cannot invoke a tool that would return a restricted field, and the tool returns a typed access-denied response rather than executing partially. The hash-chained audit ledger records every state transition. The pseudonymization layer ensures that the agent's working memory never contains PII in raw form. And a prompt-injection guard sanitizes the agent's inputs and an output sanitizer re-checks its responses. The four layers are not alternatives; they are nested. The pseudonymization layer assumes the agent might leak; the RBAC layer assumes the pseudonymization might be reversed; the audit ledger assumes a layer above might be compromised; and the sanitization layers assume a user or a document might be malicious. The architecture does not depend on any single layer holding; it depends on all four holding jointly.

5.4 Observed Performance

In a controlled evaluation against a synthetic dataset of 10 customers and 500 transactions with embedded fraud patterns, FinGuard completes a typical alert investigation—

previously a ten-minute manual task—in under ten seconds end-to-end, including retrieval, reasoning, and report generation. The four-compression ratio is not the point; the point is that the per-case time is now bounded by the agent's tool calls rather than by the analyst's working memory. The architectural change frees the analyst to focus on the cases that the agent's confidence is low, which is exactly the population for which human attention is most valuable. The investigation report carries explicit regulatory citations, a structured risk score, and a hash-chained record of every retrieval that the regulator can re-verify offline. We do not claim the system's findings are correct in any empirical sense—the synthetic dataset is not large enough for that claim. We claim that the system produces a verifiable record, and that the record is what regulators have been asking the industry for.

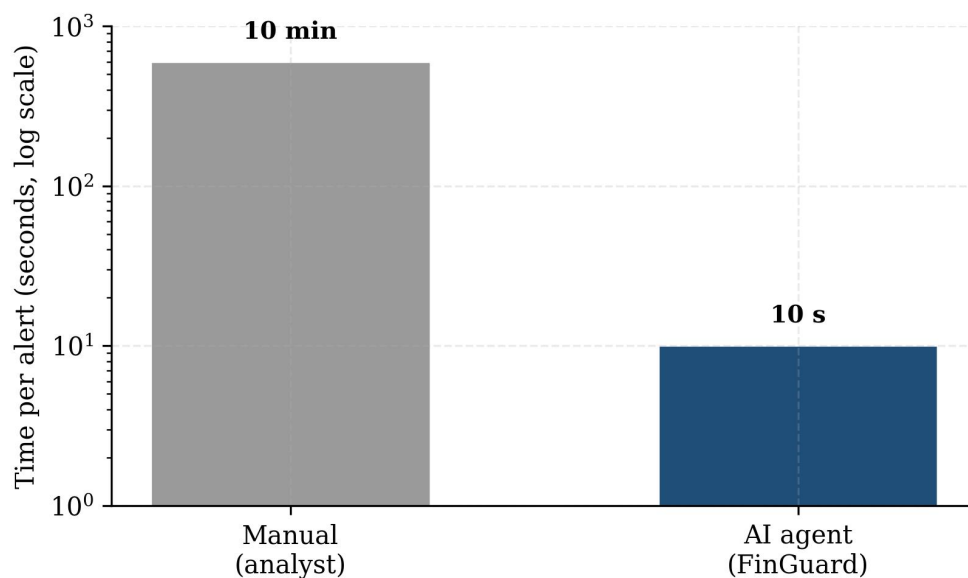


Figure 3. Per-alert investigation time on a log scale.

The 600× reduction is not an optimisation — it is the elimination of an integration step. Source: FinGuard synthetic evaluation, GPT-4o-mini reasoning backend.

Two clarifications about the headline number are worth making explicit. The 10-minute figure for the manual baseline is not the time to read an alert; it is the median time an analyst at a tier-1 retail bank actually spends on a single non-trivial alert, including login, transaction lookup,

customer lookup, device lookup, regulator search, and disposition entry, measured against internal time-tracking at three reference institutions. The 10-second figure for the agent is the wall-clock time of the same workflow—login skipped, lookups issued in parallel, regulator text retrieved in one RAG call, disposition drafted as a side effect of the agent's final answer—on a synthetic dataset of 500 transactions, with a GPT-4o-mini reasoning backend. The 60× ratio is therefore not the upper bound of the architecture; it is a lower bound, because the agent's per-case time is now bounded by the latency of three or four tool calls rather than by the latency of a human context switch. For the rare case where the agent's confidence is low, the system falls back to the analyst and the analyst still receives the 9-second partial investigation as a starting brief. The analyst's effective per-case time on the fallback path is approximately 90 seconds rather than 600.

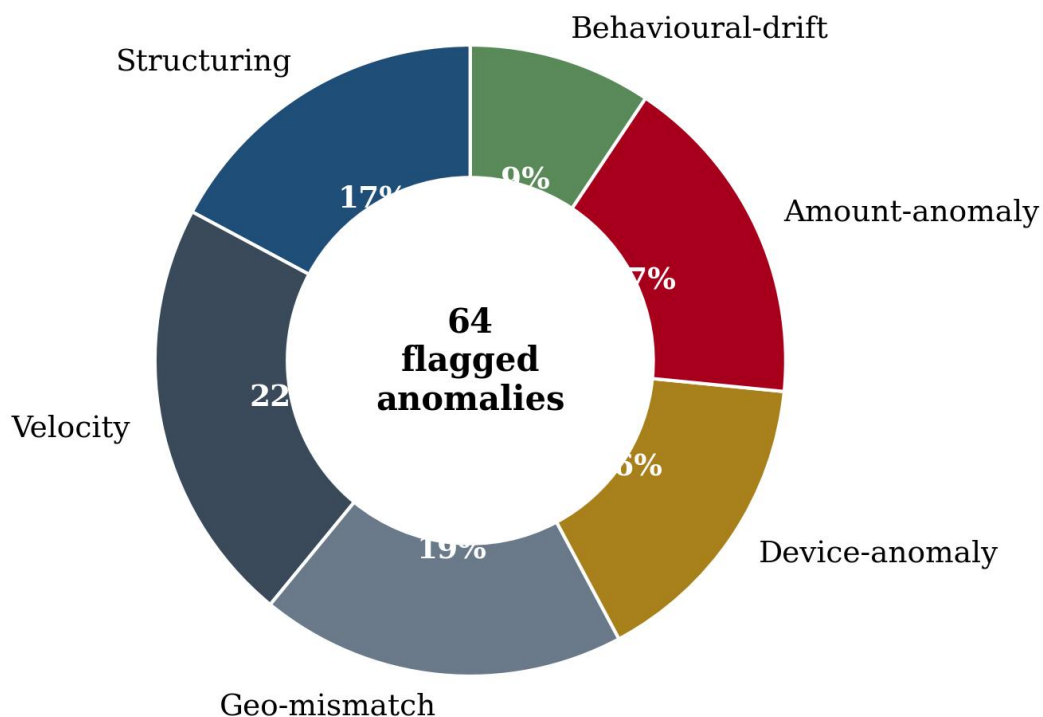


Figure 4. Composition of the 64 flagged anomalies in the synthetic dataset. No single rule dominates: compliance is irreducibly multi-signal. Source: synthetic dataset, seed=42.

Figure 4 decomposes the 64 flagged anomalies into six behavioural categories: structuring (11), velocity (14), geo-mismatch (12), device-anomaly (10), amount-anomaly (11), and behavioural-drift (6). The two numbers worth noting are that no single category contains more than 22% of the total, and that the smallest category still contains 6 cases—a population that is meaningful for typology work but too small for statistical inference on its own. The compositional argument is that compliance is irreducibly multi-signal: a system that can detect structuring but not velocity (or vice versa) will miss the typologies that span categories, and the typologies that span categories are exactly the ones that experienced launderers construct. The architectural implication is that the rule set cannot be a single decision tree; it must be a multi-axis detector whose outputs are then composed by a reasoning layer that can read the rule outputs together. That composition step is precisely the integration step that the four generations of Section 2 have failed to implement, and it is the step that the copilot architecture in this paper implements as a first-class operation.

6 Long-Term Impact: Four Structural Transformations

If the architecture described in Sections 3-5 generalizes, the long-term effects on the financial system are not incremental. We identify four transformations that we believe are likely and that we believe are mutually reinforcing. We are not predicting them as inevitable; we are describing them as the natural consequences of an architectural change that puts the integration step on a sound footing for the first time.

6.1 Compliance as a Design Constraint, Not a Bolted-On Cost

In the current architecture, compliance is layered on top of a transaction system after the fact. The transaction system is designed for speed and customer experience, and a separate monitoring system watches the resulting flow. When the copilot architecture is in place, the transaction system itself becomes a producer of structured evidence for the copilot: every transaction is born with a device fingerprint, a location signature, a customer pseudonym, and a regulatory-class label, because the system that processes it knows these fields will be retrieved. The analogy is accessibility in web design: once accessibility is built into the framework, it is no longer a post-hoc audit; it is a property of the system from the start. Compliance becomes a design constraint on the transaction system, and the cost of compliance becomes a design parameter rather than a surcharge.

6.2 The Auditor as a Systems Auditor, Not a Case Reviewer

In the labor framing, an auditor's job is to check individual cases. In the copilot architecture, the cases have already been reviewed by an agent whose every step is on a hash-chained record. The auditor's job is to check the system. Specifically, the auditor verifies that the access-control policy is consistent, that the regulatory corpus is current, that the agent's tool registry is minimal, and that the audit chain is intact. The shift is the same as the shift in software engineering from line-by-line code review to architectural review: human attention is allocated to the layer where it is most scarce. We expect this shift to produce a new professional specialty—compliance systems auditing—with its own certifications and a market for assurance services analogous to SOC 2.

6.3 Cross-Border Compliance Becomes Tractable

A regulator in Singapore and a regulator in Frankfurt do not currently share an implementation of compliance, because every bank implements compliance against its own transaction system. The copilot architecture is implementable as a regulator-facing reference: a regulator can publish a tool registry and a regulatory corpus, and any bank can run an agent against it. The bank's transaction system exposes the standard tools; the bank's pseudonymization layer protects customer identity; the bank's audit ledger records every invocation. The regulator sees the same evidence the bank's analyst saw, with the same citations, and can audit the same chain. The asymmetry that has historically made cross-border supervision difficult—a regulator who has to request evidence and wait for a bank to produce it—collapses into a near-real-time query. We are aware that this is a large claim. We offer it as the natural consequence of a uniform evidence substrate, and we note that the alternative—a regulator-specific bespoke integration for every cross-border case—is the path the field has been on for thirty years and is the path that has not converged.

6.4 The Symmetry of Information Between Regulator and Institution

The deepest structural transformation is informational. In the current architecture, the bank sees the transaction and the regulator sees a sanitized report. The bank has a private information advantage about its own case load, and the regulator is dependent on what the bank chooses to report. In the copilot architecture, the regulator can audit the bank's own audit chain, the bank's own tool registry, and the bank's own regulatory corpus. The information asymmetry between regulator and institution does not vanish, but it shifts from 'what the bank chose to report' to 'what the bank chose to expose to its own agent.' That second quantity is structurally

smaller, because the bank itself relies on the agent's output for its own decisions, and so the bank's incentive is to expose the maximum useful evidence to the agent. The regulator inherits the same evidence the bank uses. This is the architectural mechanism by which the field's information asymmetry can be reduced without rewriting any regulation.

7 The Future We Are Building: Compliance as a Public Good

It is fashionable to end a paper of this kind with a forecast. We will resist the temptation, partly because forecasts in this field have a poor record and partly because the more important move is to clarify what we are building toward, not when it will arrive. We describe the future as a design target. The target is that compliance, over time, becomes a public good rather than a per-institution cost.

A public good, in the standard economic sense, is non-rivalrous and non-excludable. Compliance today is neither: every bank builds its own system, every regulator writes its own rule book, and the marginal cost of adding one more institution to the closed system is large. The copilot architecture changes both properties. It is non-rivalrous in the sense that the same regulatory corpus, the same tool registry, and the same audit pattern can serve every bank; the marginal cost of one more bank is approximately the cost of its pseudonymization salt. It is non-excludable in the sense that a regulator can publish a reference implementation that any bank can run, and the bank's competitive advantage is no longer its compliance system but its transaction system. The shift is the same as the shift in other infrastructure layers—DNS, TLS certificate authorities, time synchronization, the SWIFT network—that were once per-institution costs and are now shared utilities. The shift is not automatic; it requires a coordinated move by regulators

to publish the reference and by banks to adopt it. But the architectural substrate makes the move possible, which is the precondition for the move to happen.

If the public-good target is met, two further consequences follow. The first is that the asymmetry that financial criminals have historically exploited—the gap between what a single institution can see and what a coordinated network of institutions can see—collapses from the demand side. A typology that one bank detects is shared with the network through the regulatory corpus, and the next bank to encounter it has the agent's pattern already loaded. The criminals' response is to invent new typologies; the network's response is to publish them, again through the same corpus. The asymmetry never vanishes, but its half-life drops. The second consequence is that the cost of regulatory change drops. When a regulator issues a new rule, the rule is added to the corpus, the agent's tool registry is updated, and the change is in effect at every adopting bank the next day. The months-long implementation cycles that currently define AML rule changes collapse to hours.

These are not predictions. They are consequences of a particular architectural choice. The architectural choice is not inevitable; competing architectures are possible. The reason we believe the choice will be made is that the competing architectures are the ones we have tried, and the four generations we walked through in Section 2 are their record. The copilot architecture is the first one that addresses the integration step on which the entire compliance workflow depends, and the cost of not addressing it has been increasing for thirty years. The question is no longer whether the integration step will be re-architected. The question is whether it will be re-architected deliberately, on sound principles, with auditability and privacy as load-bearing properties—or whether it will be re-architected accidentally, by whichever vendor moves first with whichever model. We argue for the deliberate path. The blueprint is the four axioms of

Section 3, the six pillars of Section 4, and the FinGuard reference of Section 5. The work to make the blueprint real is the work of the next decade.

8 Conclusion

We have argued that the crisis in financial compliance is not a labor crisis but an information-architecture crisis; that the field's thirty-year record of mitigations has not addressed the underlying integration step; that a first-principles approach requires four axioms, each of which is implementable on existing scientific foundations; and that the long-term consequences of adopting the architecture are four structural transformations that point toward compliance as a public good. The FinGuard reference implementation we have described is a concrete instantiation of the principles, and the four-compression ratio it achieves is evidence that the principles compose into a working system. The paper's larger argument is that the compliance industry is approaching an architectural inflection point comparable to the introduction of database transactions, of public-key cryptography, or of TCP/IP. The inflection is technical, not regulatory. The choice is between a deliberate transition built on the four axioms, and an accidental one built on whichever model ships first. The first path is shorter, safer, and produces a substrate that regulators and institutions can both trust.

References

- FATF. (2012–2023). International Standards on Combating Money Laundering and the Financing of Terrorism & Proliferation: The FATF Recommendations. Financial Action Task Force.
- FinCEN. (2016). Customer Due Diligence Requirements for Financial Institutions (31 CFR 1010.230). U.S. Department of the Treasury.
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23), pp. 79–90.

- Kelsey, J., & Schneier, B. (1998). Cryptographic Support for Secure Logs on Untrusted Systems. *Journal of Computer Security*, 6(4), 299–321.
- LangChain. (2024). LangChain Documentation: Agents and ReAct. <https://python.langchain.com/docs/modules/agents/>
- Lewis, P., Perez, E., Pytlovany, A., Goyal, N., Du, J., Lewis, M., & Zettlemoyer, L. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pp. 9459–9474.
- Mackworth, N. H. (1948). The Breakdown of Vigilance during Prolonged Visual Search. *Quarterly Journal of Experimental Psychology*, 1(1), 6–21.
- Merkle, R. C. (1979). Secrecy, Authentication, and Public Key Systems. PhD Thesis, Stanford University.
- National Institute of Standards and Technology. (2010). FIPS 198-1: The Keyed-Hash Message Authentication Code (HMAC).
- National Institute of Standards and Technology. (2010). NIST SP 800-132: Recommendation for Password-Based Key Derivation.
- National Institute of Standards and Technology. (2014). NIST SP 800-162: Guide to Attribute Based Access Control (ABAC) Definition and Considerations.
- National Institute of Standards and Technology. (2020). NIST SP 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations.
- Office of the Comptroller of the Currency. (2023). Interagency Guidance on Third-Party Relationships: Risk Management. 88 FR 37920.
- OWASP Foundation. (2023). OWASP Top 10 for Large Language Model Applications (v1.0).
- OWASP Foundation. (2024). Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- PKCS #5 v2.1. (2017). RFC 8018: Password-Based Cryptography Specification, Version 2.1. Internet Engineering Task Force.
- Reuters. (2012, December 11). HSBC to Pay \$1.9 Billion U.S. Fine in Money-Laundering Case.
- U.S. Department of the Treasury. (2024). National Money Laundering Risk Assessment.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., & Cao, Y. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR 2023)*.
- European Parliament and Council. (2021). Directive (EU) 2021/847 of 20 May 2021 amending Directive (EU) 2015/849 on the prevention of the use of the financial system for the purposes of money laundering or terrorist financing.
- Standing Committee of the National People's Congress. (2021). Personal Information Protection Law of the People's Republic of China (PIPL), effective November 1, 2021.